AI Security Engineer Multi-Engine Pack - Beginner's Build Report (A1 -> A7)

Product: Sentinel Stacks
Role agent: AI Security Engineer
Tool ID: scan_ai_security_pack
Main file: ai_ai_security_pack.py
Final version: 0.7.0-a7
Status: Hands COMPLETE (10 / 10 engines active) + Remediation mapped (43 / 43 findings)
Lab repo: ibkbalo/secops-pipeline-lab
Report date: 2026-08-04

---

1. Start here - what is this?

Imagine you hire an AI Security Engineer to check whether your company's LLM product is safe. That person does not run just one tool. They look at prompt injection, leaked API keys, RAG data leaks, agent tool abuse, MCP permissions, model supply chain, and more.

Sentinel Stacks built that job as software: the AI Security Engineer Hands Pack.

Plain English	Technical name
One AI security check	A single scanner (e.g. prompt-inject
A full AI Security Engineer review	Multi-engine pack - 10 specialized c
The written result	TOOL_STANDARDS JSON report - machine
A fix kit for each problem	Remediation Sentinel - configs, Terr

What this phase achieved: We went from an empty shell (A1) to a complete 10-engine pack (A6) and then wired every finding to an automatic fix map (A7). You can demo the whole flow offline - no live LLM target required.

---

2. Why we built it this way

2.1 The problem with "prompt-injection-only" demos

Many AI security projects stop at a single script: "we detect jailbreaks." Real AI Security Engineers cover many surfaces across the LLM lifecycle. Our pack follows the same pattern as the DevSecOps, Cloud, and Security Engineer packs already in this repo:

1. Hands first - deterministic scanners that always produce the same JSON shape.
2. Brain later - an AI agent that reads findings and decides next steps.
3. Face last - a GUI dashboard (not built yet).

This report covers Hands + Remediation mapping only.

2.2 Design rules (simple version)

Rule	What it means for you
Enterprise bar	We keep adding engines until the rol
Offline mock mode	Run python ai_ai_security_pack.py mo
Stable IDs	Every issue has a permanent ID like
Optional live tools	If you install gitleaks, semgrep, tr

---

3. The 10 engines - what each one checks

Think of each engine as a specialist on the AI Security Engineer team:

Engine key	Code	Beginner question it answers
prompt_injection	PI	Can attackers override the system pr
llm_api_keys	KEY	Are OpenAI/Anthropic (etc.) API keys
rag_data_leakage	RAG	Can one tenant see another's docs? I
output_filtering	OUT	Are model replies checked for PII, s
agent_tool_abuse	AGT	Can the agent run shell, fetch any U
mcp_permissions	MCP	Are MCP servers over-privileged with
model_supply_chain	MSC	Are model weights attested, signed,
training_poison	POI	Is fine-tune data trusted, reviewed,
model_governance	GOV	Is abuse monitored? Eval gate before
inference_hardening	INF	Is the inference API authenticated,

Finding ID format (locked forever):

```
AISEC-{ENGINE_CODE}-{NNN}
```

Examples: AISEC-PI-001, AISEC-RAG-003, AISEC-INF-002.

---

4. How it fits together (architecture)

```
You run the pack
       |
       v
ai_ai_security_pack.py
       |
       +-- Reads mock fixture  OR  live target path
       +-- Runs 10 engine workers (PI, KEY, RAG, ...)
       +-- Merges findings + severity scores
       +-- Outputs TOOL_STANDARDS JSON
                |
                v
       ai_remediation_engine.py  (A7)
                |
                +-- Looks up each AISEC-* ID in FIX_MAP
                +-- Generates Terraform / configs / runbooks
```

```
                +-- Zips a hardening kit (dry_run mode)
```

Key files:

File	Purpose
ai_ai_security_pack.py	The pack - all 10 engines
mock_ai_security_vulnerable.json	Fake "bad" LLM app - 43 findings
mock_ai_security_clean.json	Fake "good" LLM app - 0 findings
tool_registry.json	Catalog entry for orchestrators
ai_remediation_engine.py	Turns findings into fix kits (v1.5.0

---

5. Phase-by-phase - what we built

We delivered in 7 phases (A1 -> A7). Each phase turned on more engines and raised the finding count on the vulnerable mock.

5.1 Rollup table

Phase	Version	What turned on	Active engines	Vuln mock findings
A1	0.1.0-a1	Registry + fixtures only (stubs)	0 / 10	0
A2	0.2.0-a2	Prompt injection (PI) + LLM API keys	2 / 10	8
A3	0.3.0-a3	+ RAG leakage (RAG), Output filter	4 / 10	17
A4	0.4.0-a4	+ Agent tools (AGT), MCP permissions	6 / 10	26
A5	0.5.0-a5	+ Model supply chain (MSC), Training	8 / 10	34
A6	0.6.0-a6	+ Governance (GOV), Inference harden	10 / 10	43
A7	0.7.0-a7	FIX_MAP for all 43 AISEC-* IDs	10 / 10	43 mapped -> 0 unmapped

Pack completion: pack_hands_complete: true at A6. Remediation bar closed at A7.

5.2 A1 - Pack skeleton

Goal: Define the shape of the product before real checks.

* Created ai_ai_security_pack.py with engine registry, ID scheme, backend detection, JSON merge runner.
* Added mock fixture files (vulnerable + clean profiles).
* Registered scan_ai_security_pack in tool_registry.json.
* All engines started as stubs (registered but inactive) - 0 findings by design.

Why it mattered: Later phases only *activate* engines; they do not rewrite the whole pack.

5.3 A2 - Prompt injection + LLM API keys

Goal: Can someone jailbreak the model or steal provider keys?

* PI (5): System prompt not isolated, indirect injection via docs, jailbreak filter off, plus unblocked sample payloads.
* KEY (3): OpenAI/Anthropic keys in tracked files; secret scanning disabled in CI.

5.4 A3 - RAG leakage + output filtering

Goal: Is retrieval isolating data, and are replies guarded?

* RAG (5): Cross-tenant retrieval, PII in vector index, ACLs not enforced, sensitive HR/legal paths indexed.
* OUT (4): No PII redaction, toxic filter off, no code/secret exfil guard, max tokens not enforced.

5.5 A4 - Agent tools + MCP permissions

Goal: Can agents and connectors expand blast radius?

* AGT (5): Unrestricted web fetch, shell tool on, empty allowlist, no SSRF protection, exfil via tool args.
* MCP (4): Over-privileged filesystem/browser servers, no least privilege, inventory undocumented.

5.6 A5 - Model supply chain + training poison

Goal: Are models and training data trustworthy?

* MSC (4): No provenance attestation, untrusted hub weights, plugin signing off, no model SBOM.
* POI (4): Untrusted fine-tune data, no provenance review, poison detection off, no human review gate.

5.7 A6 - Governance + inference hardening (hands complete)

Goal: Close the last gaps and hit 100% pack completion.

* GOV (4): No abuse monitoring, logging retention disabled, no eval gate before prod, AUP not enforced.
* INF (5): No auth, public anonymous endpoint, no rate limits, no cost alerts, TLS not required.

Final vulnerable mock breakdown (43 total):

Code	Count	Theme
PI	5	Prompt injection, jailbreaks, isolat
AGT	5	Agent tool abuse / SSRF / exfil
RAG	5	Cross-tenant, PII, ACL, sensitive pa
INF	5	Inference API auth, rate limits, TLS
OUT	4	Output guardrails
MCP	4	MCP permission sprawl
MSC	4	Model provenance / supply chain
POI	4	Training / fine-tune poisoning
GOV	4	Abuse monitoring & deploy gates
KEY	3	LLM provider API key exposure

Clean mock: 0 findings, all 10 engines report passed checks.

5.8 A7 - Remediation mapping (fix kits)

Goal: Every AISEC-* finding gets a known fix - not just a description.

* Bumped ai_remediation_engine.py to v1.5.0.
* Added 43 FIX_MAP entries for all AI Security Engineer pack IDs.
* New config templates for AI security fixes (prompt isolation, RAG, agent/MCP, output, supply chain, training,

governance, inference).
* Smoke test: pack mock -> remediation engine -> 43 mapped, 0 unmapped, hardening kit ZIP generated.

What you get after A7: Scan -> structured JSON -> downloadable kit with Terraform snippets, configs, and YAML runbooks.

---

6. Try it yourself (copy-paste)

Run these from the repo root. Works on Windows PowerShell or any shell with Python 3.

6.1 Run the vulnerable mock scan

```
cd C:\DevSecOps-Lab\secops-pipeline-lab
python ai_ai_security_pack.py mock
```

You should see JSON with 43 findings, version 0.7.0-a7, and "pack_hands_complete": true.

6.2 Save output and run remediation

```
python ai_ai_security_pack.py mock | Out-File -Encoding utf8 .tmp_aisec_vuln.json
python ai_remediation_engine.py .tmp_aisec_vuln.json
```

Look for mapped: 43 and unmapped: 0 in the remediation summary. A kit ZIP appears under hardening_kits/ (gitignored).

6.3 Run the clean mock (sanity check)

```
python ai_ai_security_pack.py . mock_ai_security_clean.json
```

Expect 0 findings - proves engines only fire when evidence exists.

Windows tip: Always use Out-File -Encoding utf8 when saving JSON. PowerShell's default > redirect uses UTF-16 and breaks json.load.

---

7. How this compares to other Sentinel role packs

Role pack	Tool ID	Engines	Mock findings (vuln)	Remediation IDs
AI Security Engineer	scan_ai_security_pack	10	43	AISEC-* (43 mapped)
Security Engineer	scan_security_engineer_pack	10	53	PERIM-* (53 mapped)
DevSecOps	scan_devsecops_pack	10	62	DEVSEC-*
Cloud Security	scan_cloud_pack	multi	170	CLOUD-*

All four share the same TOOL_STANDARDS JSON contract and the same Hands -> Brain -> Face roadmap.

---

8. What comes next (not in this report)

Step	Status	Plain English
AI Security Engineer Hands (A1-A6)	Done	All 10 scanners work
AI Security Engineer Remediation (A7	Done	Every finding has a fix path
All four role Hands + Remediation	Done	DevSecOps, Cloud, SE, AI Sec
Role Brain	Planned	AI agent orchestrates scans + triage
Face (GUI dashboard)	Planned	Visual ops console

There is no GUI yet. Output is JSON (+ remediation ZIP) until the Face phase.

---

9. One-page cheat sheet

```
Product ............. AI Security Engineer Multi-Engine Pack
Module .............. ai_ai_security_pack.py
Version ............. 0.7.0-a7
Engines ............. 10 active / 0 stub
Mock vuln findings ... 43
Mock clean findings .. 0
ID scheme ........... AISEC-{CODE}-{NNN}
Remediation engine ... ai_remediation_engine.py v1.5.0
FIX_MAP AISEC entries  43 (0 unmapped)
Hands complete ....... YES (A6)
Remediation mapped ... YES (A7)
Next ................ Brain -> Face
```

---

10. Glossary for beginners

Term	Simple definition
LLM	Large language model - the AI that a
Prompt injection	Tricking the model into ignoring its
RAG	Retrieval-augmented generation - the
MCP	Model Context Protocol - connectors
Finding	One specific security problem with I
Engine	A focused checker inside the pack (e
Fixture / mock	Fake data file so you can demo witho
FIX_MAP	Lookup table: finding ID -> fix temp
Hardening kit	ZIP bundle of generated fix files fr
TOOL_STANDARDS	Shared JSON schema all Sentinel scan

Sentinel Stacks ? AI Security Engineer Hands A1-A7 ? Beginner build report